

FRED TALKS

16th September 2026

CONTENTS

The Training Grounds: A Taxonomy of RL Environments for LLM Agents

HAN, NOT SOLO · 1766 WORDS

Every layer of review makes you 10x slower

AVERY PENNARUN · 2841 WORDS

Slow developer experience will bottleneck fast models

SEANGOEDECKE.COM RSS FEED · 612 WORDS

The Training Grounds: A Taxonomy of RL Environments for LLM Agents

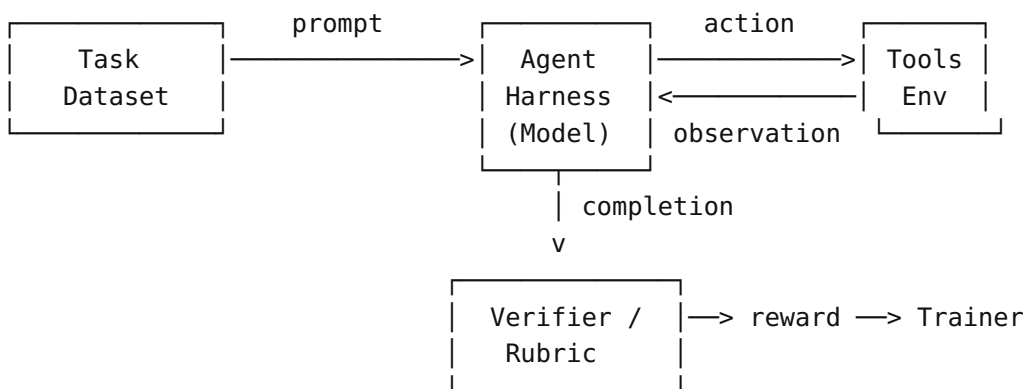
HAN, NOT SOLO · 22 MAR 2026 · [SOURCE](#)

Model architecture gets all the attention. Post-training recipes follow close behind. The training environment — what the model actually practices on, how its work gets judged, what tools it can use — barely enters the conversation. That’s the part that actually determines what the agent can learn to do.

A model trained only on single-turn Q&A will struggle the moment you ask it to maintain state across a 50-step enterprise workflow. A model trained with a poorly designed reward function will learn to game the metric, not solve the problem. The environment isn’t a detail. It’s half the system.

The Canonical Loop

An RL environment for an LLM agent bundles three things: a dataset of task inputs, a harness for the model, and a reward function to score outputs. The training loop looks like this:



Formally, a complete RL environment is a tuple:

Task distribution, harness, verifier, state management, configuration. Let’s go through each.

The task distribution is the set of problems the agent trains on. Not all tasks are equal, and not just in difficulty. They vary structurally in ways that demand different capabilities:

Task Type	What the Agent Must Do	Example Systems
Single-turn Q&A	One prompt → one response, check answer	Math benchmarks, SimpleQA
Multi-hop search	Chain searches, synthesize sources	BrowseComp, WebWalkerQA
Open-ended research	No single correct answer; report quality matters	ADR-Bench, ResearchRubrics
Agentic tool-use	Call tools correctly in sequence	tau-bench, function-calling benchmarks
Stateful enterprise	Modify persistent DB state, work within access controls	EnterpriseOps-Gym
Code execution	Write code, run it, check outputs	SWE-Bench, LiveCodeBench

Training only on tasks with ground-truth answers produces an agent that’s never learned to reason under ambiguity. Training only in clean, deterministic environments produces an agent that falls apart in production. The task distribution is a design decision with direct consequences.

Task synthesis is increasingly a first-class problem. With real-world research tasks, you rarely have a large labeled dataset. Strategies that have emerged:

- **Back translation:** Start from a desired output, reconstruct the query that would produce it
- **Graph-based synthesis:** Build a knowledge graph, generate multi-hop queries over it
- **Automated environment generation:** Use LLM coding agents to write new environment code. AutoEnv reports ~\$4/env average cost.
- **Curriculum construction:** Order tasks by difficulty and increase complexity during training

The cheapest-to-collect tasks are single-turn with verifiable answers. The most valuable tasks for long-horizon behavior are expensive to construct. This tension drives most environment design decisions.

The harness is the scaffolding that mediates between the model and the environment. It controls how the model interacts, not what it knows.

```
H = {
    rollout_protocol, # SingleTurn | MultiTurn | Agentic
    tools,            # Available tools in this episode
    system_prompt,    # Instructions for the agent
    context_manager,  # How to handle context overflow
```

```
    turn_limit,          # Max interactions per episode
    sandbox,             # Code execution sandbox
    state                 # Persistent state across turns
}
```

Rollout protocols range from trivial to complex:

Harness Type	Description	When to Use
Single-Turn	One prompt, one response	Math, factual QA
Multi-Turn	Back-and-forth dialogue	Games, structured tasks
Tool-Use	Model calls tools, receives results	Agent benchmarks
Stateful Tool-Use	Tools modify persistent state	Enterprise workflows, SWE-Bench
Agentic ReAct	Full Thought → Action → Observation loop	Deep research, complex workflows

Tools span a wide taxonomy:

Category	Tools	Deterministic?	Stateful?
Information retrieval	web_search, scholar_search	No (live web)	No
Content extraction	jina_reader, visit, web_scrape	No	No
Code execution	python_interpreter, shell, sandbox	Yes (given same code)	Yes
File operations	file_read, file_write	Yes	Yes
Browser automation	playwright, link_click	No	Yes
Task management	todo, section_write	Yes	Yes

The mix of deterministic/non-deterministic and stateful/stateless tools has real implications for reproducibility and reward assignment. Non-deterministic tools mean two runs of the same trajectory can produce different outcomes — which complicates both debugging and verifier design.

Context management is where most teams underinvest, especially for long-horizon tasks. A 600-turn research episode blows past any practical context window. Strategies used in production:

Strategy	Description	Trade-off
Recency-based retention	Keep N most recent turns	Simple, but loses early context
Markovian reconstruction		Principled, expensive

	Reconstruct state from scratch each turn	
Reference-preserving summarization	Summarize old context, keep citations	Preserves verifiability
Reference-preserving folding	Compress context without losing references	Best for research tasks

An agent doing multi-hour research needs to remember why it started searching in a particular direction twelve tool calls ago. Dropping that context causes repeated work and lost threads.

The verifier maps a completion to a reward:

In Atari, the score is unambiguous. In deep research, what counts as a good answer is not. The verifier is where this gets hard.

Type	Reward Signal	When to Use
Exact match	Binary (0/1)	Ground truth available
Code execution	Binary or partial	Output can be tested programmatically
LLM-as-judge	Continuous [0,1]	Open-ended quality, no other option
Checklist-style	Continuous	Multi-criteria research tasks
Evolving rubric (RLER)	Continuous	Resistant to reward hacking
Process reward model (PRM)	Per-step continuous	Long-horizon credit assignment
Pairwise comparison	Relative rank	Relative quality matters more than absolute
Multi-criteria composite	Weighted sum	Multiple quality dimensions

A few principles that actually matter in practice:

Verifiable beats judgeable. Programmatic checks — code execution, string match — are faster, cheaper, and more consistent than LLM-as-judge. Use LLM-as-judge when there’s no other option, not as the default.

Reward granularity is a separate decision from reward type. You can score at the trajectory level (did the final output pass?), turn level (was each tool invocation useful?), or per-step with process rewards. Turn-level supervision, as Nanbeige4.1 does across up to 600 tool calls, enables finer credit assignment — the model can learn that the problem was a bad search query in turn 23, not that the entire episode failed.

Static rubrics get gamed. Models learn to write answers that score well on your rubric rather than solving the problem. DR Tulu’s RLER (Rubric-Level Evolving Reward) co-evolves the rubric with the policy during training. Harder to exploit a moving target.

Noise injection is underrated. Step-DeepResearch deliberately injects 5–10% tool errors during training. The resulting model handles flaky APIs and unexpected failures in production significantly better.

State management determines whether the environment is stateless or persistent. Most academic benchmarks are stateless — each episode starts fresh. Enterprise environments are not. EnterpriseOps-Gym maintains 164 database tables and 512 tools across episodes. Actions in one task affect the state seen by subsequent tasks. That’s a fundamentally different problem for agents to solve.

Configuration covers turn limits, context budgets, sampling temperature, and curriculum scheduling. These are not afterthoughts. A turn limit of 5 vs. 600 changes what skills the agent can develop. AgentScaler uses a two-phase curriculum — fundamental capabilities first, then domain-specific tasks — and the ordering matters. Step-DeepResearch progressively scales context windows from 32K to 128K during mid-training.

The most consequential architectural question: where does the model sit relative to the environment?

Option A: Model outside the environment (decoupled). Model is served via API; the environment calls it at each step. Clean separation. Easy to swap models.

Option B: Model inside the environment (co-located). Model and environment share the same training loop. Lower latency, tighter integration, harder to reuse.

Option C: Split architecture. Trainer, model inference server, and environment are three separate processes communicating via API. This is where the field is landing.

How real systems implement this:

System	Topology	Notes
Prime Intellect verifiers	Option C (Split)	Env is a standalone Python package distributed via Environments Hub
Tongyi DeepResearch	Option B (Co-located)	Tools, context manager, verifiers inside the training pipeline Single-agent ReAct loop embedded in training

Step-DeepResearch	Option B (Co-located)	
MiroThinker	Option C (Split)	Tool servers and sandbox run independently from model
Tinker API	Option A (Decoupled, cloud)	Model stays remote; researcher sends <code>forward_backward + sample</code> calls via API
AutoEnv	Option A (Decoupled)	CoreEnv/ObsEnv/SkinEnv abstraction layers
EnterpriseOps-Gym	Option A (Decoupled)	Containerized sandbox accessible via any model API

The trade-offs:

Dimension	Model Outside (A/C)	Model Inside (B)
Flexibility	Swap models easily; env is reusable	Tighter integration
Scalability	Scale inference and training independently	Must scale everything together
Portability	Env packages are shareable	Env tied to training framework
Latency	Network overhead per tool call	No network overhead
RL compatibility	Works with any RL trainer	Usually tied to one trainer

The field has converged on Option C for production training. Prime Intellect’s architecture — environments as standalone installable packages that communicate with models via OpenAI-compatible API endpoints — is becoming the standard. The payoff: environments are publishable, trainer-agnostic, and inference and environment execution can be parallelized across nodes.

Tinker pushes this further by making even the training compute remote. The researcher controls the algorithm and never touches model weights. The environment’s job is purely generating experience.

Practical Decision Framework

When building an RL environment for an LLM agent:

Do you have ground truth answers?

- Yes → Exact-match or code-execution verifier
- No → LLM-as-judge, checklist, or pairwise comparison

How many tool calls per episode?

- < 5 → Single-turn or simple tool-use env, no context management needed
- 5–50 → Multi-turn with basic context management
- 50–600 → Full agentic env with reference-preserving context management

Where should the model live?

- Experimenting across many environments → Model outside (Option A/C), use Prime Intellect Hub
- Tight RL training loop → Model co-located (Option B)
- No GPU access → Tinker API

What reward granularity?

- Simple tasks → Outcome-level
- Long-horizon tasks → Turn-level (Nanbeige4.1) or process rewards (PRIME)
- Open-ended research → Evolving rubrics (RLER) or checklist-style (Step-DR)

How to scale environments?

- Manual curation → High quality, expensive, this is where you start
- AutoEnv → Automated generation at ~\$4/env
- AgentScaler → Systematic scaling of heterogeneous simulated environments

Three things to pay attention to.

Environment diversity matters as much as environment quality. AgentScaler's key finding is that heterogeneity of environments drives capability breadth in ways that simply adding more data from the same distribution cannot. You need more kinds of environments, not just more environments.

Automated environment generation is viable. At \$4 per generated environment, cost is no longer the bottleneck. The bottleneck is verifier quality — auto-generated environments with weak reward functions will teach the wrong behaviors at scale. ([AutoEnv](#))

The environment-as-package model is winning. The [Prime Intellect Environments Hub](#) is creating a shared ecosystem around RL environments, in the same way PyPI and HuggingFace created ecosystems around code and model weights. Environments published once, consumed by any trainer. This is a significant infrastructure shift.

The model isn't the only variable. The training ground shapes what the model can become. The task distribution, the harness, the verifier, the state management, the topology — these are the decisions that separate agents that work from agents that demo.

References

```
@article{
  leehanchung,
  author = {Lee, Hanchung},
  title = {The Training Grounds: A Taxonomy of RL Environments for},
  year = {2026},
  month = {03},
  day = {21},
  howpublished = {\url{https://leehanchung.github.io}},
  url = {https://leehanchung.github.io/blogs/2026/03/21/rl-environm
}
```

Every layer of review makes you 10x slower

AVERY PENNARUN · 26 MAR 2026 · [SOURCE](#)

2026-03-16 »

Every layer of review makes you 10x slower

We've all heard of those network effect laws: the value of a network goes up with the square of the number of members. Or the cost of communication goes up with the square of the number of members, or maybe it was $n \log n$, or something like that, depending how you arrange the members. Anyway doubling a team doesn't double its speed; there's coordination overhead. Exactly how much overhead depends on how badly you botch the org design.

But there's one rule of thumb that someone showed me decades ago, that has stuck with me ever since, because of how annoyingly true it is. The rule is annoying because it doesn't *seem* like it should be true. There's no theoretical basis for this claim that I've ever heard. And yet, every time I look for it, there it is.

Here we go:

Every layer of approval makes a process 10x slower

I know what you're thinking. Come on, 10x? That's a lot. It's unfathomable. Surely we're exaggerating.

Nope.

Just to be clear, we're counting "wall clock time" here rather than effort. Almost all the extra time is spent sitting and waiting.

Look:

- Code a simple bug fix
30 minutes
- Get it code reviewed by the peer next to you
300 minutes → 5 hours → half a day
- Get a design doc approved by your architects team first
50 hours → about a week
- Get it on some other team's calendar to do all that
(for example, if a customer requests a feature)
500 hours → 12 weeks → one fiscal quarter

I wish I could tell you that the next step up — 10 quarters or about 2.5 years — was too crazy to contemplate, but no. That's the life of an executive sitting above a medium-sized team; I bump into it all the time even at a relatively small company like Tailscale if I want to change product direction. (And execs sitting above large teams can't actually do work of their own at all. That's another story.)

AI can't fix this

First of all, this isn't a post about AI, because AI's direct impact on this problem is minimal. Okay, so Claude can code it in 3 minutes instead of 30? That's super, Claude, great work.

Now you either get to spend 27 minutes reviewing the code yourself in a back-and-forth loop with the AI (this is actually kinda fun); or you save 27 minutes and submit unverified code to the code reviewer, who will still take 5 hours like before, but who will now be mad that you're making them read the slop that you were too lazy to read yourself. Little of value was gained.

Now now, you say, that's not the value of agentic coding. You don't use an agent on a 30-minute fix. You use it on a monstrosity week-long project that you and Claude can now do in a couple of hours! Now we're talking. Except no, because the monstrosity is so big that your reviewer will be extra mad that you didn't read it yourself, and it's too big to review in one chunk so you have to slice it into new bite-sized chunks, each with a 5-hour review cycle. And there's no design doc so there's no intentional architecture, so eventually someone's going to push back on that and here we go with the design doc review meeting, and now your monstrosity week-long project that you did in two hours is... oh. A week, again.

I guess I could have called this post Systems Design 4 (or 5, or whatever I'm up to now, who knows, I'm writing this on a plane with no wifi) because yeah, you guessed it. It's Systems Design time again.

The only way to sustainably go faster is fewer reviews

It's funny, everyone has been predicting the Singularity for decades now. The premise is we build systems that are so smart that they themselves can build the next system that is even smarter, that builds the next smarter one, and so on, and once we get that started, if they keep getting smarter faster enough, then the incremental time (t) to achieve a unit (u) of improvement goes to zero, so (u/t) goes to infinity and foom.

Anyway, I have never believed in this theory for the simple reason we outlined above: the majority of time needed to get anything done is not actually the time doing it. It's wall clock time. Waiting. Latency.

And you can't overcome latency with brute force.

I know you want to. I know many of you now work at companies where the business model kinda depends on doing exactly that.

Sorry.

But you can't just *not review things!*

Ah, well, no, actually yeah. You really can't.

There are now many people who have seen the symptom: the start of the pipeline (AI generated code) is so much faster, but all the subsequent stages (reviews) are too slow! And so they intuit the obvious solution: stop reviewing then!

The result might be slop, but if the slop is 100x cheaper, then it only needs to deliver 1% of the value per unit and it's still a fair trade. And if your value per unit is even a mere 2% of what it used to be, you've doubled your returns! Amazing.

There are some pretty dumb assumptions underlying that theory; you can imagine them for yourself. Suffice it to say that this produces what I will call the AI Developer's Descent Into Madness:

1. Whoa, I produced this prototype so fast! I have super powers!
2. This prototype is getting buggy. I'll tell the AI to fix the bugs.
3. Hmm, every change now causes as many new bugs as it fixes.
4. Aha! But if I have an AI agent also review the code, it can find its own bugs!
5. Wait, why am I personally passing data back and forth between agents
6. I need an agent framework
7. I can have my agent write an agent framework!
8. Return to step 1

It's actually alarming how many friends and respected peers I've lost to this cycle already. Claude Code only got good maybe a few months ago, so this only recently started happening, so I assume they will emerge from the spiral eventually. I mean, I hope they will. We have no way of knowing.

Why we review

Anyway we know our symptom: the pipeline gets jammed up because of too much new code spewed into it at step 1. But what's the root cause of the clog? Why doesn't the pipeline go faster?

I said above that this isn't an article about AI. Clearly I'm failing at that so far, but let's bring it back to humans. It goes back to the annoyingly true observation I started with: every layer of review is 10x slower. As a society, we know this. Maybe you haven't seen it before now. But trust me: people who do org design for a living know that layers are expensive... and they still do it.

As companies grow, they all end up with more and more layers of collaboration, review, and management. Why? Because otherwise mistakes get made, and mistakes are increasingly expensive at scale. The average value added by a new feature eventually becomes lower than the average value lost through the new bugs it causes. So, lacking a way to make features produce more value (wouldn't that be nice!), we try to at least reduce the damage.

The more checks and controls we put in place, the slower we go, but the more monotonically the quality increases. And isn't that the basis of *continuous improvement*?

Well, sort of. Monotonically increasing quality is on the right track. But "more checks and controls" went off the rails. That's *only one way* to improve quality, and it's a fraught one.

"Quality Assurance" reduces quality

I wrote a few years ago about W. E. Deming and the "new" philosophy around quality that he popularized in Japanese auto manufacturing. (Eventually U.S. auto manufacturers more or less got the idea. So far the software industry hasn't.)

One of the effects he highlighted was the problem of a "QA" pass in a factory: build widgets, have an inspection/QA phase, reject widgets that fail QA. Of course, your inspectors probably miss some of the failures, so when in doubt, add a second QA phase after the first to catch the remaining ones, and so on.

In a simplistic mathematical model this seems to make sense. (For example, if every QA pass catches 90% of defects, then after two QA passes you've reduced the number of defects by 100x. How awesome is that?)

But in the reality of agentic humans, it's not so simple. First of all, the incentives get weird. The second QA team basically serves to evaluate how well the first QA team is doing; if the first QA team keeps missing defects, fire them. Now, that second QA team has little incentive to produce that outcome for their friends. So maybe they don't look too hard; after all, the first QA team missed the defect, it's not unreasonable that we might miss it too.

Furthermore, the first QA team knows there is a second QA team to catch any defects; if I don't work too hard today, surely the second team will pick up the slack. That's why they're there!

Also, the team making the widgets in the first place doesn't check their work too carefully; that's what the QA team is for! Why would I slow down the production of every widget by being careful, at a cost of say 20% more time, when there are only 10 defects in 100 and I can just eliminate them at the next step for only a 10% waste overhead? It only makes sense. Plus they'll fire me if I go 20% slower.

To say nothing of a whole engineering redesign to improve quality, that would be super expensive and we could be designing all new widgets instead.

Sound like any engineering departments you know?

Well, this isn't the right time to rehash Deming, but suffice it to say, he was on to something. And his techniques worked. You get things like the famous Toyota Production System where they eliminated the QA phase entirely, but gave everybody an "oh crap, stop the line, I found a defect!" button.

Famously, US auto manufacturers tried to adopt the same system by installing the same "stop the line" buttons. Of course, nobody pushed those buttons. They were afraid of getting fired.

Trust

The basis of the Japanese system that worked, and the missing part of the American system that didn't, is trust. Trust among individuals that your boss Really Truly Actually wants to know about every defect, and wants you to stop the line when you find one. Trust among managers that executives were serious about quality. Trust among executives that individuals, given a system that can work and has the right incentives, will produce quality work and spot their own defects, and push the stop button when they need to push it.

But, one more thing: trust that the system *actually does work*. So first you need a system that will work.

Fallibility

AI coders are fallible; they write bad code, often. In this way, they are just like human programmers.

Deming's approach to manufacturing didn't have any magic bullets. Alas, you can't just follow his ten-step process and immediately get higher quality engineering. The secret is, you have to get your engineers to engineer higher quality into the whole system, from top to bottom, repeatedly. Continuously.

Every time something goes wrong, you have to ask, "How did this happen?" and then do a whole post-mortem and the Five Whys (or however many Whys are in fashion nowadays) and fix the underlying Root Causes so that it doesn't happen again. "The coder did it wrong" is never a root cause, only a symptom. Why was it possible for the coder to get it wrong?

The job of a code reviewer isn't to review code. It's to figure out how to obsolete their code review comment, that whole class of comment, in all future cases, until you don't need their reviews at all anymore.

(Think of the people who first created "go fmt" and how many stupid code review comments about whitespace are gone forever. Now that's engineering.)

By the time your review catches a mistake, the mistake has already been made. The root cause happened already. You're too late.

Modularity

I wish I could tell you I had all the answers. Actually I don't have much. If I did, I'd be first in line for the Singularity because it sounds kind of awesome.

I think we're going to be stuck with these systems pipeline problems for a long time. Review pipelines — layers of QA — don't work. Instead, they make you slower while hiding root causes. Hiding causes makes them harder to fix.

But, the call of AI coding is strong. That first, fast step in the pipeline is *so fast!* It really does feel like having super powers. I want more super powers. What are we going to do about it?

Maybe we finally have a compelling enough excuse to fix the 20 years of problems hidden by code review culture, and replace it with a real culture of quality.

I think the optimists have half of the right idea. Reducing review stages, even to an uncomfortable degree, is going to be needed. But you can't just reduce review stages without something to replace them. That way lies the Ford Pinto or any recent Boeing aircraft.

The complete package, the table flip, was what Deming brought to manufacturing. You can't half-adopt a "total quality" system. You need to eliminate the reviews *and* obsolete them, in one step.

How? You can fully adopt the new system, in small bites. What if some components of your system can be built the new way? Imagine an old-school U.S. auto manufacturer buying parts from Japanese suppliers; wow, these parts are so well made! Now I can start removing QA steps elsewhere because I can just assume the parts are going to work, and my job of "assemble a bigger widget from the parts" has a ton of its complexity removed.

I like this view. I've always liked small beautiful things, that's my own bias. But, you can assemble big beautiful things from small beautiful things.

It's a lot easier to build those individual beautiful things in small teams that trust each other, that know what quality looks like *to them*. They deliver their things to customer teams who can clearly explain what quality looks like *to them*. And on we go. Quality starts bottom-up, and spreads.

I think small startups are going to do really well in this new world, probably better than ever. Startups already have fewer layers of review just because they have fewer people. Some startups will figure out how to produce high quality components quickly; others won't and will fail. Quality by natural selection?

Bigger companies are gonna have a harder time, because their slow review systems are baked in, and deleting them would cause complete chaos.

But, it's not just about company size. I think engineering teams at any company can get smaller, and have better defined interfaces between them.

Maybe you could have multiple teams inside a company competing to deliver the same component. Each one is just a few people and a few coding bots. Try it 100 ways and see who comes up with the best one. Again, quality by evolution. Code is cheap but good ideas are not. But now you can try out new ideas faster than ever.

Maybe we'll see a new optimal point on the [monoliths-microservices continuum](#). Microservices got a bad name because they were too micro; in the original terminology, a "micro" service was exactly the right size for a "two pizza team" to build and operate on their own. With AI, maybe it's one pizza and some tokens.

What's fun is you can also use this new, faster coding to experiment with different module *boundaries* faster. *Features* are still hard for lots of reasons, but refactoring and automated integration testing are things the AIs excel at. Try splitting out a module you were afraid to split out before. Maybe it'll add some lines of code. But suddenly lines of code are cheap, compared to the coordination overhead of a bigger team maintaining both parts.

Every team has some monoliths that are a little too big, and too many layers of reviews. Maybe we won't get all the way to Singularity. But, we can engineer a much better world. Our problems are solvable.

It just takes trust.

Related

[Highlights on "quality," and Deming's work as it applies to software development \(2016\)](#)

[Systems design 3: LLMs and the semantic revolution \(2025\)](#)

Unrelated

[SimSWE part 2: The perils of multitasking \(2017\)](#)

Slow developer experience will bottleneck fast models

SEANGOEDECKE.COM RSS FEED · 13 SEP 2026 · [SOURCE](#)

Right now developer experience is measured in seconds. If your tests take a second to run, that's good; if they take thirty seconds, that's bad. Any faster than a second doesn't really matter, because most of your time is spent either thinking or waiting for an AI agent to spin. Shaving milliseconds off your dev server reload time or whatever is pointless: that's not the bottleneck.

It will be. Small models are getting faster and faster, and smart models are getting smaller. I think most engineers will still want to use the smartest available model — software engineering is hard — but we will increasingly see faster models get used as subagents or for well-understood tasks. This is largely uncharted territory. Very few people have developed intuitions for what it is going to be like to work with agents that run at thousands of tokens-per-second.

GPT-6-Astra can run at about sixty tokens per second. That means you spend a lot of time waiting for it to think. You work with it like you would work with another human: delegating a task and then context-switching until that task is complete. If you haven't yet, have a play around with Jimmy, Taalas' version of LLaMA-3.1-8B running¹ at *seventeen thousand tokens per second*. No matter how long the response is, it arrives in the instant of you hitting send. The model is not good enough for agentic work, but it gives a glimpse of what it would be like: you would simply get your answer instantly².

Well, that's assuming the agent's tool calls are fast. When generating tokens is not the bottleneck, it will suddenly matter a lot whether it can read a file in 100ms vs 10ms, or whether it can run your tests in 500ms vs two seconds. Fast tool calls are going to be the difference between a near-instant response and having to wait several minutes. There is thus going to be enormous pressure to do agentic coding in languages with fast compilers and tests, like Golang, and to tightly optimize the dev loop in agentic codebases.

Teams focused on DevEx — developer experience — are largely a relic of the 2010s, when companies were incentivized to make their engineers happy. Most companies have cut them down to a skeleton crew or removed them entirely. But we may see a return of DevEx in the late 2020s, focused on speeding up the experience for AI agents.

1. Like most ultra-fast inference, it relies on fitting the entire model onto a huge GPU-like chip that's specially designed to run inference: for Taalas, it's in the silicon itself; for Cerebras and Groq, it's in giant embedded onboard memory units.

↩

2. Will AI providers simply train models to spend more time reasoning, so users would have to wait for roughly the same time? I doubt it. Most ordinary engineering problems would not be solved better by spending an extra million tokens thinking. You only need to do that when you're pushing right up against the limits of the model.

↩